CS-233 Project - Milestone 2: Predicting Gaming Addiction

Tommaso Sommaruga (407010) Steven Ji (392693) Tom Ravot (392844)

1 Introduction

We predict both a continuous addiction score and a discrete addiction level from gaming habits. The dataset contains 2000 samples with 13 features, split into 1600 training examples and 400 test examples. We study regression and classification, and implement an MLP for the supervised tasks as well as K-Means for a clustering baseline.

For classification, we use a Softmax output with weighted multiclass CrossEntropy, optional polynomial feature expansion, class-aware oversampling, and class weights to handle imbalance. We also reuse the best grid-search batch size during the final retrain.

2 Method

2.1 Multi-Layer Perceptron (MLP)

2.1.1 Regression

Our baseline is a single-hidden-layer MLP (32 units, ReLU) trained via mini-batch SGD (lr $= 10^{-3}$, batch size 8) using MSE loss, Gaussian initialization, and z-score standardized inputs. This configuration serves as the reference point for all subsequent performance comparisons

2.1.2 Classification

For classification, we use a softmax output layer with cross-entropy loss. Since the classes are strongly imbalanced, we combine class-weighted loss with feature engineering and, in some runs, simple oversampling of the minority classes. Macro-F1 is used as the main metric because it reflects performance on all classes more fairly than accuracy.

2.2 K-Means Clustering

K-Means is an unsupervised clustering algorithm that partitions data into K groups by iteratively updating cluster centers. To adapt it for classification, each cluster is assigned a label via **majority voting**: the most frequent true label among points in the cluster becomes the cluster's label. At inference, a new point is assigned to its nearest cluster and receives that cluster's label.

Our implementation initializes K centers by randomly sampling from the training data. At each iteration, points are assigned to their nearest center using Euclidean distance, then centers are recomputed as the mean of their assigned points. The algorithm stops when the total center displacement falls below 10^{-6} or `max_iters` is reached. Empty clusters retain their previous center to avoid degeneracy.

3 Experiments & Results

3.1 Multi-Layer Perceptron (MLP)

3.1.1 Regression

3.1.1.1 Feature Selection

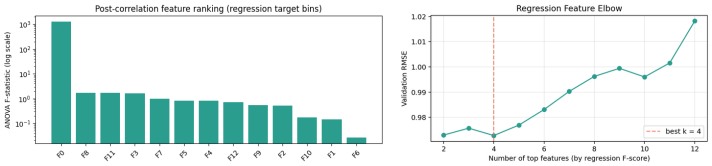


Figure 1: F-score Feature Ranking and Elbow Test

The ANOVA F-statistic ranking (left) highlights F0 as the dominant feature, with remaining predictors showing a significantly lower impact. To determine the optimal input subset, we performed an 'elbow test' on the validation RMSE (right). The metric is minimized at $k = 4$; performance degrades with the inclusion of additional features, confirming that they introduce more noise than signal. We therefore select the top four features to ensure model simplicity and robustness.

3.1.1.2 Optimizer

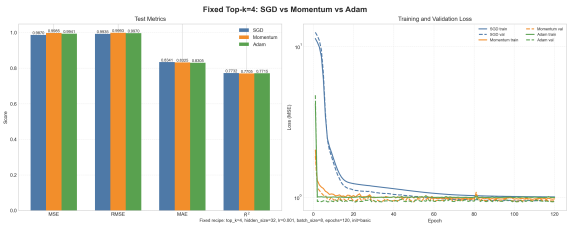


Figure 2: Optimizer comparison

The optimizer comparison shows that all three methods reach similar final performance, but Adam converges significantly faster than SGD and basic Momentum. Following the initial drop, the loss curves remain nearly flat, characterized by minor oscillations rather than a sustained upward trend, which indicates neither overfitting nor underfitting. The model demonstrates a robust generalization capability, thereby rendering further optimization unnecessary.

3.1.1.3 Final improvements

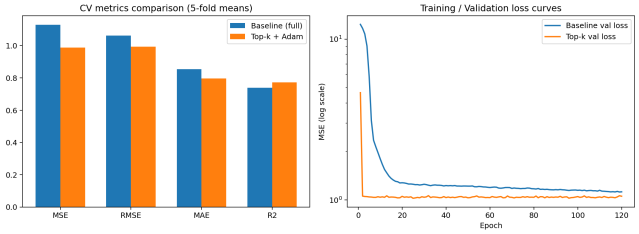


Figure 3: Optimizer comparison

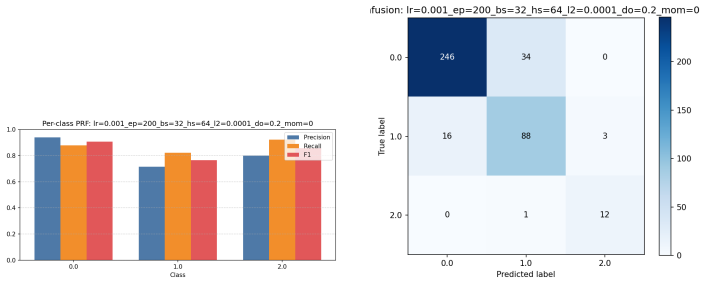
The final model architecture consists of a single-hidden-layer MLP optimized with the Adam optimizer and feature selection via ANOVA F-statistic ($k = 4$: [0, 11, 4, 8]). This configuration yields marked improvements in predictive metrics and significantly accelerates training, converging in just 21 epochs compared to the baseline's 119.

3.1.2 Classification

Classification suffered from class imbalance. We ran a focused set of experiments combining polynomial feature expansion (degree 2), a variance→correlation→ANOVA top-k filter, class-weighted cross-entropy, random oversampling, L2, dropout, and small architecture/optimizer sweeps.

Hyperparameters were chosen by 5-fold CV and grid-search over lr $\in \{1 \times 10^{-5},\ 1 \times 10^{-4},\ 1 \times 10^{-3}\}$, epochs $\in \{100,\ 200\}$, batch $\in \{16,\ 32\}$, hidden $\in \{32,\ 64,\ 128\}$; the repeated best was lr=1e-3, epochs=200, batch_size=16, hidden_size=128. The final model (retrained on the full training set) achieved Macro-F1 ≈ 0.78 and test accuracy $\approx 85\%$.

Class weights plus oversampling gave the largest F1 gains; polynomial expansion plus ANOVA improved minority separability; modest L2/dropout stabilized results across folds.



(a) Per-class Prec. / Rec. / F1 (b) Confusion matrix (test)

Figure 4: Per-class PRF and confusion matrix

3.1.3 Final Results on Test Set

Method	Acc (%)	Macro-F1	MSE
MLP Reg.	–	–	0.987
MLP Classif.	86.5	0.8434	–

3.2 K-Means Clustering

3.2.1 Elbow Analysis.

We evaluated $K \in \{3, 6, 9, 12, 15, 18, 21\}$ using the elbow method on inertia. As shown in Figure 5, a clear elbow appears at $K = 6$, indicating diminishing geometric gains beyond this point.

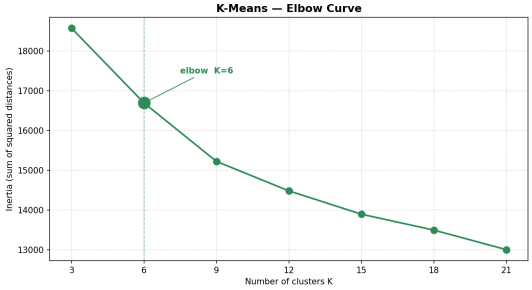


Figure 5: K-Means elbow curve - elbow at $K = 6$.

3.2.2 Accuracy and Macro-F1 vs. K.

Figure 6 shows that the best test accuracy is reached at $K = 9$ (76.0%), while Macro-F1 peaks at $K = 21$ (0.583). This divergence reflects class imbalance: larger K creates dedicated clusters for minority classes, boosting recall and F1 at the cost of accuracy.

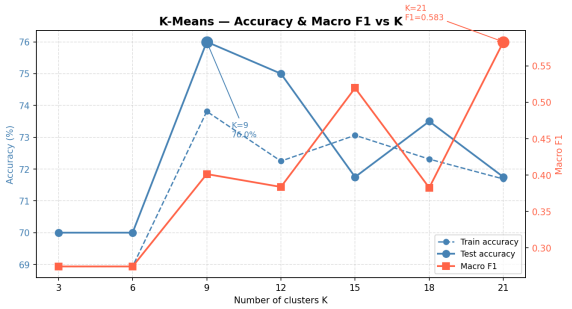


Figure 6: Test accuracy and Macro-F1 as a function of K.

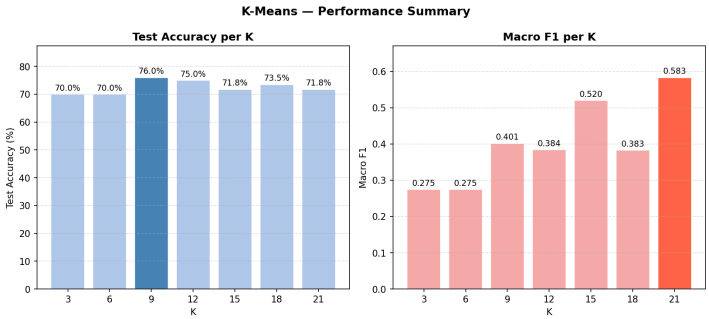


Figure 7: Test accuracy (left) and Macro-F1 (right) per K.

3.2.3 Final Configuration.

We select $\mathbf{K = 21}$ as our final configuration to maximize Macro-F1 (0.583). Despite a slight drop in test accuracy (71.8%), this choice ensures fair minority class representation.

K	Test Acc (%)	Macro-F1
6 (elbow)	70.0	0.275
9 (best acc)	76.0	0.401
21 (best F1)	**71.8**	**0.583**

4 Discussion & Conclusion

The strongest result is the final MLP classifier, which reaches 86.5% test accuracy and a Macro-F1 of 0.8434. This is a large improvement over the earlier baseline and clearly better than K-Means. Even when K-Means is optimized for minority classes at K = 21 (achieving a Macro-F1 of 0.583), the MLP classifier significantly outperforms it. The Macro-F1 gain matters most because it reflects minority-class performance rather than only majority-class accuracy.

The main improvement came from refining the classification setup to better match the multiclass and imbalanced nature of the task, using a Softmax output with weighted multiclass cross-entropy and explicit imbalance handling. In practice we added class-aware oversampling, class weights, polynomial feature expansion before feature selection, and a tuned regularized MLP with Xavier initialization and Adam optimization, plus L2 and dropout. Xavier is the weight initialization rule, while Adam is the optimizer; together they made training more stable and easier to tune. The grid-search/final-retrain fix mainly ensured that the best configuration was reused consistently, while L2 and dropout provided smaller stabilization benefits.

The regression model stayed comparatively simple but effective, with MSE 0.987 and $R^2 = 0.772$ in cross-validation. Compared with the classification task, regression benefited more from light feature selection than from heavier architectural changes. Overall, the experiments show that the best results came from matching the modeling choice to the task: imbalance-aware training for classification, and a compact MLP with selected features for regression.